# Kani is Hackable

# Tweak everything to your specifications

- Essential agent primitives & features:

- Chat history management

- Robust function calling

- Model-agnostic

- Multimodal inputs

- Async-first design

11

# kani (カニ)



kani (カニ) is a lightweight and highly hackable framework for chat-based language models with tool usage/function calling.

Compared to other LM frameworks, kani is less opinionated and offers more fine-grained customizability over the parts of the control flow that matter, making it the perfect choice for NLP researchers, hobbyists, and developers alike.

kani comes with support for the following models out of the box, with a **model-agnostic** framework to add support for many more:

- OpenAI Models (`pip install "kani[openai]"`)
- Anthropic Models (`pip install "kani[anthropic]"`)
- Google AI Models (`pip install "kani[google]"`)
- and _every_ chat model available on Hugging Face through `transformers` or `llama.cpp`! (`pip install "kani[huggingface]"`)

## Quickstart

kani requires Python 3.10 or above. To install model-specific dependencies, kani uses various extras (brackets after the library name in `pip install`). To determine which extra(s) to install, see the model table, or use the `[all]` extra to install everything.

First, install the library. In this quickstart, we'll use the OpenAI engine, though kani is model-agnostic.

```
$ pip install "kani[openai]"
```

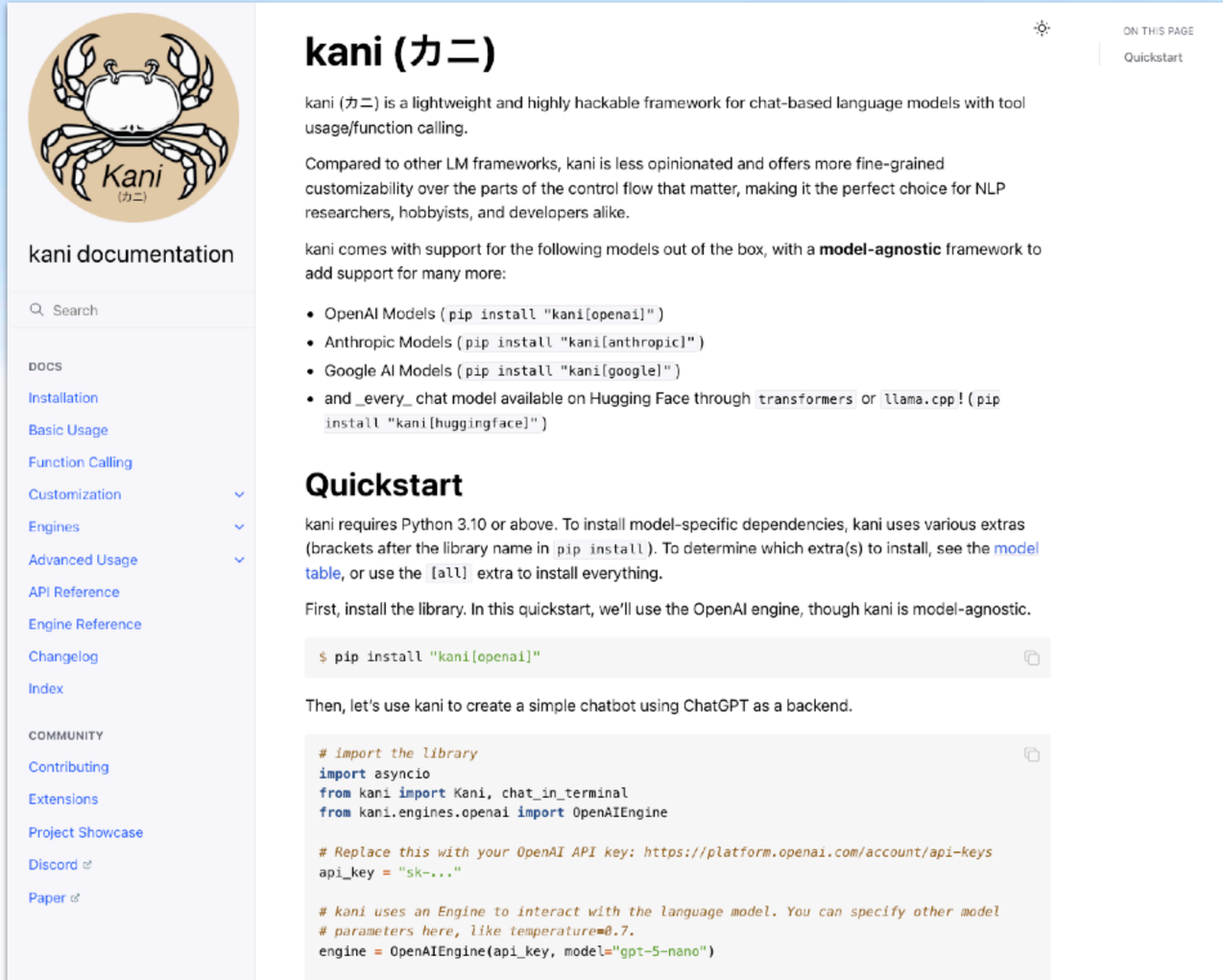
Then, let's use kani to create a simple chatbot using ChatGPT as a backend.

```
# import the library
import asyncio
from kani import Kani, chat_in_terminal
from kani.engines.openai import OpenAIEngine

# Replace this with your OpenAI API key: https://platform.openai.com/account/api-keys
api_key = "sk-..."

# kani uses an Engine to interact with the language model. You can specify other model
# parameters here, like temperature=0.7.
engine = OpenAIEngine(api_key, model="gpt-5-nano")
```

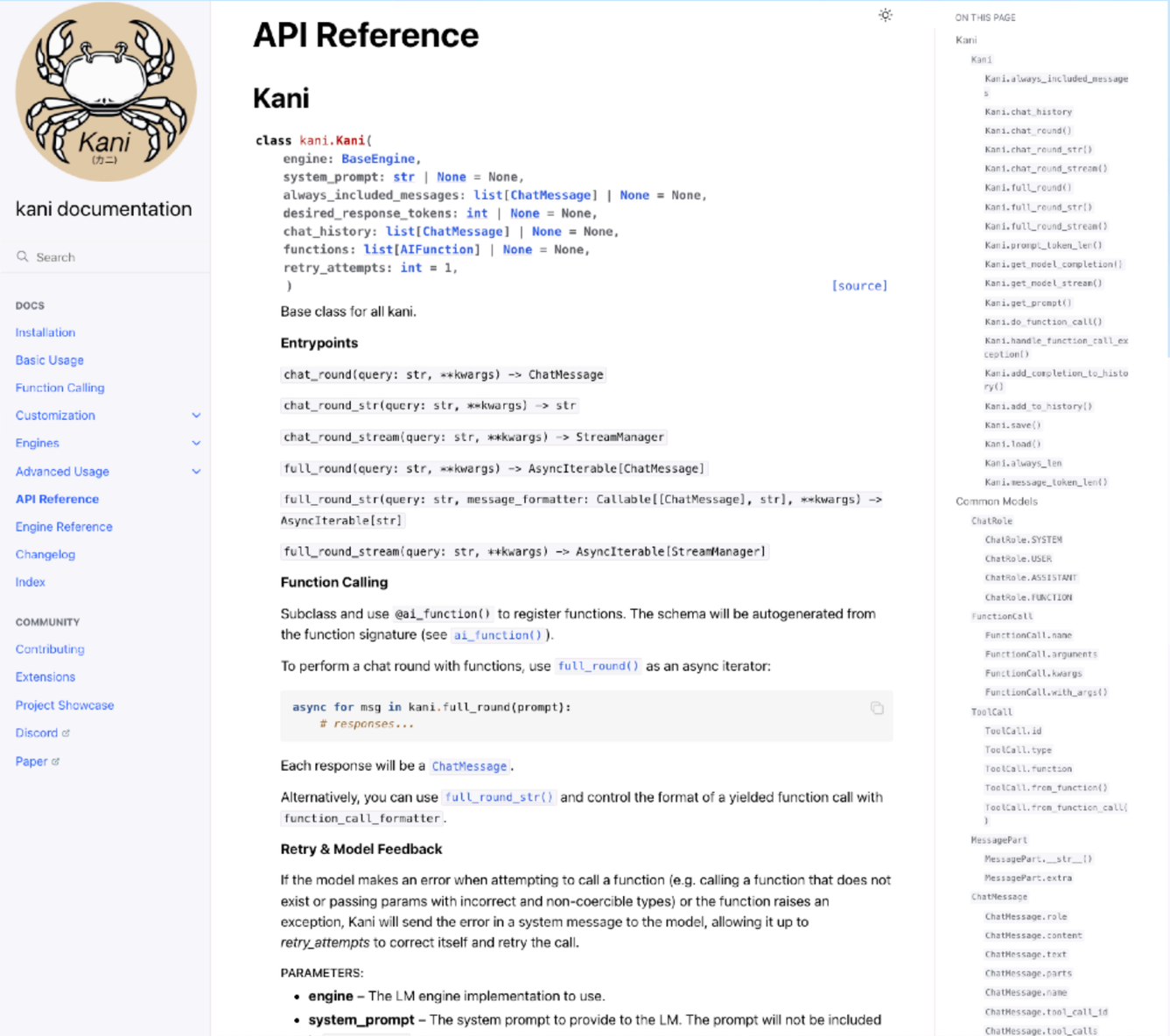
# Kani is Hackable

## Tweak everything to your specifications

- Essential agent primitives & features:
  - Chat history management
  - Robust function calling
  - Model-agnostic
  - Multimodal inputs
  - Async-first design



# Kani has extensive documentation & testing

**https://kani.readthedocs.io**

- More examples on how to:
  - Override prompt building
  - Provide few-shot prompts
  - Host Kani online
  - Implement new LLM engines
  - ...and more!
- We test Kani with real LLMs on every commit!